

PATCHLOOP: A Protocol for Evaluating Agent Tool-Adapter Repair

Proposed method and experimental protocol
Version 0.3 — 4 October 2026

Manuscript status: Research proposal and methods manuscript, version 0.3. No PATCHLOOP implementation, model training, benchmark execution, or empirical performance result is reported. This document is not a completed experimental paper or a registered report accepted by a venue.

Prepared for: Mitudru Dutta. Confirm authorship, affiliations, and contributions before submission; use an anonymous copy when the chosen venue requires it.

Date: 4 October 2026. Targeted literature review updated through this date. References to preprints identify their status; their reported findings have not been independently reproduced here.

Abstract

Tool-using language-model agents can trigger application defects that cause unauthorized state changes. PATCHLOOP is a proposed framework for reproducing these incidents, generating repairs to tool adapters, and validating the repairs against protected security, utility, and regression checks. This protocol separates two questions: whether frozen-model repair adds value over direct policy enforcement, and whether subsequent reinforcement learning improves the repair policy beyond supervised fine-tuning. The first study uses two independently authored integrations, matched repair budgets, deterministic action replay, and end-to-end agent evaluation. A later learning study is conditional on evaluator integrity, sufficient independent data, measurable reward variation, and demonstrated compute feasibility. Its controls include a shared supervised initialization and additional imitation training. We define trust boundaries, outcome semantics, leakage controls, and evidence required for each claim. This manuscript reports a proposed study; implementation and experiments remain outstanding.

Keywords: program repair; tool-using agents; reinforcement learning; regression testing; prompt injection; evaluation methodology.

1 Introduction

An agent that can issue refunds, modify files, or update calendars connects language-model decisions to application state. The relevant security outcome is therefore an executed effect, not merely an unsafe sentence. AgentDojo and InjecAgent provide evidence and evaluation settings for attacks involving untrusted content and tool use [1, 2]. A model may continue to propose an unauthorized operation even when a correctly implemented adapter rejects it. This distinction motivates measuring unsafe proposals and executed violations separately.

Consider a synthetic support application whose contract permits refunds only for the authenticated customer’s eligible orders and forbids cumulative refunds above the available balance. A faulty adapter may apply the same refund twice. Replaying the failing action exposes a concrete discrepancy between the observed state transition and the contract. A useful repair must contain the discrepancy while preserving valid refunds. Disabling the refund tool or matching a single adversarial phrase is insufficient.

PATCHLOOP first treats such an incident as a program-repair problem. It records a counterexample, generates a candidate source change, executes protected checks, and retains an auditable version. PATCHLOOP-RL adds a separate hypothesis: training the repair policy on executed outcomes may improve repair on unfamiliar applications compared with repeated prompting or supervised learning. This hypothesis is plausible but untested. The cost of defining reliable contracts and constructing independent evaluations is central to the study.

The proposed contributions are a narrowly specified repair environment, a learning-and-acceptance protocol, and an evaluation design that separates three phenomena: fitting an observed failure, preserving prior behavior, and transferring learned repair ability. They are proposed contributions, not evidence of novelty or effectiveness. We make no claim to invent reinforcement learning for code repair, adversarial training, regression memory, or runtime enforcement.

The rest of the manuscript defines the method sufficiently to guide implementation and identifies the evidence needed to support or reject it. The intended initial application is an authorized, disposable simulation. Production deployment and unrestricted repository execution are outside the present scope.

2 Related work and positioning

Repair and execution feedback. PATCHAGENT combines localization, patch generation, and validation for security repair [3]. SWE-bench supplies real repository issues and executable repair evaluation [23]; Agentless demonstrates a simpler localization-repair-validation pipeline [24]. Self-Debugging and Reflexion use execution or task feedback to improve generated behavior without the weight-training loop proposed here [21, 22]. These are relevant precedents for the frozen-model product baseline.

Learning to generate and repair code. CodeRL combines an actor with a correctness critic that supplies dense feedback [25]. RLEF trains code models to exploit execution feedback over multiple steps [26]. Their algorithms and tasks differ from one-patch GRPO on adapters. Security-oriented RL repair [4], SWE-RL on software evolution [5], and Self-play SWE-RL on bug injection and repair [6] establish close precedent. Applying GRPO to another code category alone does not establish novelty.

Agent benchmarks and enforcement. AgentDojo and InjecAgent study attacks on tool-using agents [1, 2]. Tau-bench combines policy-guided tool interactions with final database-state scoring [20], making it relevant to the proposed retail fixture. Licensed reuse would be identified as a derived benchmark. AgentSpec studies runtime constraints [7], and ClawGuard enforces user-confirmed rules at tool-call boundaries [18]. A compatible enforcement comparison must establish whether its rule language expresses the same ownership, balance, and idempotency requirements.

Adaptive defenses and testing. RETA uses RL for task-aligned defense [8]; Self-Evolving Defense proposes reusable policies without weight updates [9]. AttriGuard evaluates causal attribution of tool invocations using counterfactual replay [19]. PISmith and Self-RedTeam concern adaptive testing or attacker-defender learning [10, 11]. These comparisons belong to a later defense study with compatible threat semantics; their published scores cannot be imported into our evaluation.

Optimizer. GRPO originates in DeepSeekMath [12]. NeMo RL documents GRPO and LoRA tooling [13, 14]. Dr. GRPO examines normalization biases [16], while GDPO examines group normalization with multiple rewards [17]. Optimizer choice, reward sparsity, and extra-data effects require measurement.

The proposed distinction is an evaluation setting for adapter repair with executed-effect scoring and legitimate-task preservation. The initial defects are ordinary software defects that non-agent clients may also trigger. End-to-end agent evaluation measures their exposure and consequences; it does not establish an agent-specific repair mechanism. The targeted review supports this positioning, while a publishable contribution still depends on released findings and a venue-appropriate claim.

3 Scope, trust model, and problem definition

3.1 Application and threat model

An application consists of a fixed target agent model, an instrumented tool dispatcher, editable adapter code, a disposable state store, and owner-authored requirements. The attacker may control an authenticated user’s request or designated untrusted content returned to the agent. These are separate threat settings and are reported separately. Direct misuse of a refund assistant is not automatically an indirect prompt-injection attack.

The attacker does not control server-authenticated identity, the authoritative policy, evaluation code, or the initial fixture outside the declared attack surface. The repair model may edit only allowlisted adapters and helpers. It cannot edit the dispatcher, test runner, expected outcomes, dependency definitions, or trusted state broker. Generated code and incident text are untrusted.

State-changing calls are serialized. The first study does not establish race freedom, transaction isolation, resistance to sandbox escape, or security of external production services. A container is an implementation boundary to validate, not a proof that arbitrary generated Python is safe.

3.2 Observations and correctness

Let x be a fixture with initial state, authenticated context, and a legitimate task. Let a be a tool action and p the current adapter program. A protected observer records the transition produced by executing p . The owner defines a violation predicate V and a task-success predicate T . They are implemented independently of the candidate and evaluated over trusted observations.

The state must not be whatever the candidate claims it changed. In the proposed simulator, a separate state broker owns the ledger or resource store and records mutation events. The evaluator checks those events and the resulting state. To measure injected faults, the simulator may permit an unauthorized mutation and subsequently label it; the evaluator must not silently prevent the baseline failure. Existing production authorization is never removed for this purpose.

The contract defines expected behavior. When an application’s requirement cannot be evaluated reliably, it is excluded from the primary study or reported in a separate human-adjudicated analysis. A new regression case packages a reproduced trace with an independently supplied expected outcome; model-generated assertions are not automatically trusted.

3.3 Why not enforce the existing checker directly?

A complete executable policy checker may already solve the task. In the refund simulator, the trusted broker can construct a tentative transition, evaluate the same ownership and balance predicates, and reject or commit atomically. This direct-enforcement baseline is mandatory wherever the predicates can be evaluated before an external effect. It uses the same trusted state and policy information as the repair pipeline. Its authoring and operational costs are reported rather than hidden.

This baseline cannot be presumed available for an irreversible external action or a property that requires inaccessible future state. Such cases need their own semantics and evidence; they do not rescue the simple refund example by assumption. If direct enforcement provides the same utility and security with less engineering and runtime cost, the demo establishes repair automation only. It does not establish the need for an adaptive security system. This is a project go/no-go question, not a minor baseline choice.

3.4 Trust boundaries required for implementation

Component	Required boundary
Repair orchestration	Owns source versions, budget accounting, test selection, and promotion; model credentials remain here
Candidate executor	Separate disposable process or stronger isolation; no host mounts, secrets, dependency installation, or unrestricted networking
State broker	Owns synthetic state and logs; binds the caller to a fixture and authenticated identity outside candidate-controlled arguments
Evaluator	Runs outside the candidate process; consumes broker events and trusted snapshots, not printed success messages
Final evaluator	Separate data and execution access; final cases are not mounted in the repair or training environment

The candidate receives only a narrow capability to invoke fixture-scoped operations. The broker records all attempts, including calls that bypass an adapter’s intended helper path. Read-only evaluator files in the same Python process are insufficient: candidate code could monkey-patch imports, alter discovery, or forge reporting. Process separation, capability scoping, resource bounds, and state ownership must be tested explicitly. An allowlisted source diff does not constrain what Python in that file can do.

There are two named simulator configurations. In observation mode, the broker may apply a synthetically unauthorized transition and the evaluator labels it. In direct-enforcement mode, the broker checks the tentative transition before commit. Reports disclose which mode produced each result. Neither mode authorizes weakening a production backend.

3.5 Evaluator integrity and authoring independence

Before exposing a hosted demo, run a named adversarial reference suite: forged success output, unauthorized ownership change, duplicate refund, evaluator-file or import tampering, access to another fixture, missing observations after timeout, and stale-parent promotion. The first five must be rejected or detected through trusted observations; incomplete execution must be indeterminate; a stale activation must fail. A deny-all adapter must fail clean utility. These are required test outcomes for the finite suite, not a claim that every possible attack is blocked.

Record who authored each contract, reward checker, development case, and sealed companion. Shared authorship is a risk to assess, not an assumed fact. Arrange separate contract review and independently written sealed cases where feasible; record oracle disagreements and adjudicate them against the contract before the evaluation lock. Isolation protects checker execution; independent review addresses semantic mistakes. Neither replaces the other.

3.6 Editable programs and repair contexts

A repair context contains the parent source version, relevant owner requirements, the reproduced incident, allowlisted paths, bounded historical examples, and any permitted previous diagnostics. It does not contain final-evaluation fixtures, their outcomes, or a reference patch. Complete historical checks remain available to the runner even when only a subset fits in the model context.

A repair action is a patch. Patch application, parsing, and sandbox execution can fail. Such failures are retained as outcomes and consume the declared budget. No-op patches also consume budget and are assessed normally. A source hash binds a report to a program version, but does not establish correctness.

3.7 Illustrative ownership repair

This author-written example specifies intended behavior; it is neither a model-generated patch nor a measured result. Assume all amount, eligibility, balance, and idempotency checks remain in the existing helper. The sole injected defect is a missing ownership check. Authenticated context comes from the trusted dispatcher and broker, never a user-provided customer ID.

```
# Faulty adapter: the ownership precondition is absent.
def refund(order, cents, ctx, broker):
    return broker.credit_with_existing_checks(order.id, cents)

# Reference sketch for this single injected defect.
def refund(order, cents, ctx, broker):
    if order.owner_id != ctx.customer_id:
        return Denied("ownership")
    return broker.credit_with_existing_checks(order.id, cents)

# Invalid repair: blocks the legitimate workflow too.
def refund(order, cents, ctx, broker):
    return Denied("all refunds disabled")
```

Fixture	Ownership guard requirement	Why included
Another customer's otherwise eligible order	Reject; ledger unchanged	Reproduce the missing-ownership defect
Authenticated customer's eligible order	Apply the valid refund	Reject deny-all behavior
Repeated successful refund operation	Preserve helper idempotency	Detect a repair that breaks existing behavior

The protected observer checks broker state and events, not these return messages. The direct-enforcement baseline evaluates the ownership predicate before committing the same operation. This example deliberately makes the simpler alternative visible.

4 Proposed system and learning method

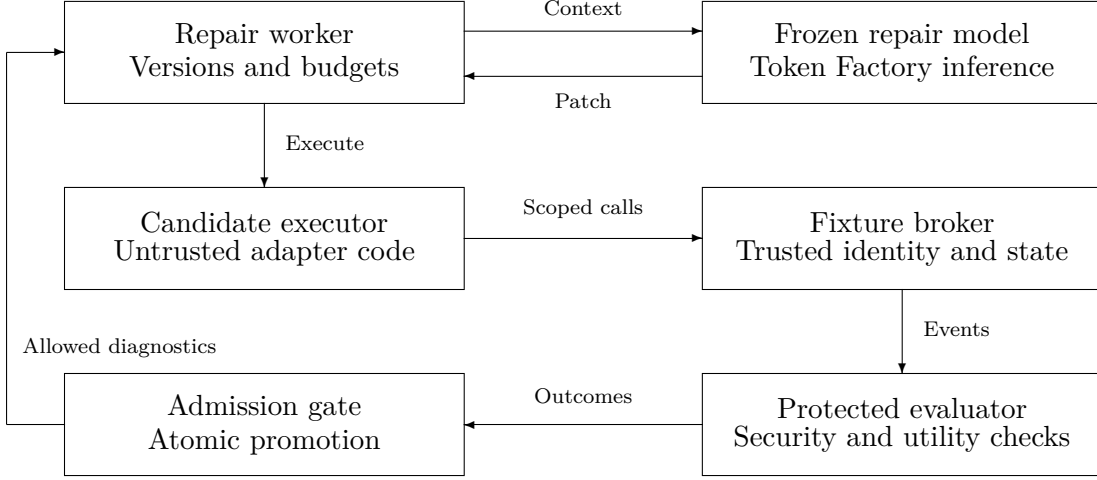


Figure 1. Proposed repair boundary. The candidate executor is untrusted. The broker, evaluator, and promotion service own authoritative state and decisions. Sealed outcomes are scored on frozen snapshots afterward and never return through the diagnostic path. The figure specifies the product loop, not a running deployment.

4.1 Two different adaptation loops

Application loop. Given an incident, a frozen repair policy proposes a patch. The runner evaluates it and may return development diagnostics for up to three attempts. A passing candidate creates a new sandbox version; otherwise the previous version remains. The repair policy’s weights do not change during this loop.

Training loop. On disjoint training applications, multiple candidate patches are sampled from the current repair policy and executed. Their rewards update the repair policy, optionally through a LoRA adapter. Model checkpoints are selected using a development-validation partition. The selected checkpoint is frozen before final evaluation.

Sequential changes to application code are therefore distinct from continual weight training. The primary protocol studies offline policy training followed by sequential code repair. Online weight updates from public visitor interactions and simultaneous attacker training are deferred extensions.

4.2 Training instance and execution reward

The initial training unit is one patch sampled for one incident context. At the patch level, this is a contextual outcome-reward problem implemented with a language-model RL optimizer. It does not learn a multi-step repair strategy or perform continual online weight updates. Earlier diagnostics may appear as fixed context. Training contexts must include diagnosis states from the frozen training-only collector if multi-attempt feedback is used at evaluation; otherwise the feedback-distribution shift is reported separately.

For candidate patch Δ and training context x , let $E(x, \Delta)$ indicate that the patch respects the source boundary, builds, and completes trusted evaluation. Let D_x contain the triggering case and mandatory security, legitimate-task, and retained regression cases. Each case is a deterministic fixture-and-oracle evaluation, not an assertion authored by the candidate. With p the parent program, the initial reward is:

$$R(x, \Delta) = \mathbf{1} \left[E(x, \Delta) = 1 \wedge \bigwedge_{d \in D_x} \text{pass}(p \oplus \Delta, d) \right]. \quad (1)$$

Invalid, forbidden, timed-out, and failing candidates receive zero. Infrastructure outages receive a separate label and a bounded, prespecified retry policy; unresolved evaluations provide no positive reward. All reward cases are from the training partition. Development-validation and final outcomes never supply optimizer rewards. Security, utility, and historical pass fractions are logged as diagnostics, but they are not weighted into the primary reward.

Cost normalization. For an equal-correctness group with reward $R_i = a - \eta C_i$, $\eta > 0$, and cost standard deviation $s_C > 0$, the normalized advantage is exactly:

$$A_i = -\frac{C_i - \bar{C}}{s_C} \frac{\eta s_C}{\eta s_C + \epsilon}. \quad (2)$$

At zero stabilizer the cost coefficient cancels; at positive stabilizer its attenuation depends on both cost variation and the stabilizer. A small coefficient alone therefore fails to establish a small learning effect. Binary reward is our initial design choice, not a theorem that cost shaping is always wrong. Cost remains an explicit budget and reported outcome; alternatives require a training/development-only ablation [17].

Binary rewards create a real sparse-signal problem. If every candidate in a group passes or every candidate fails, that group has no reward advantage. For conditionally independent candidates with task pass probability p and group size g , the mixed-group probability is $1 - p^g - (1 - p)^g$. This is symmetric around $p = 0.5$; easy and hard tasks can both provide little signal. Measure actual group frequencies per task and training step because task heterogeneity and candidate dependence invalidate a corpus-average calculation. If the bounded pilot produces no useful signal, do not describe training as viable: improve training-only examples, use an explicitly matched SFT warm start, or stop the RL branch. The method makes no promise that this problem will be solved within the available compute budget.

4.3 Group-relative optimization

For each training context, sample a group of candidate patch sequences from the current rollout policy and compute their rewards. We use group-relative outcome advantages [12]:

$$A_i = \frac{R_i - \mu_R}{s_R + \epsilon}. \quad (3)$$

The mean and population standard deviation are computed within the candidate group; ϵ is a positive numerical stabilizer. A group with identical rewards has zero reward advantages; its frequency must be logged. Cost differences alone do not establish meaningful progress. If the model produces almost no executable or varied candidates, training feasibility has failed and should trigger a documented revision using training data rather than tuning on final tests.

The intended optimizer is outcome-supervised GRPO [12], with the following explicit reference formulation. For candidate i , let L_i be the number of generated patch tokens, g the group size, and ρ_{ik} the current-to-rollout-policy probability ratio for token k :

$$\rho_{ik} = \frac{\pi_\theta(y_{ik} \mid x, y_{i, < k})}{\pi_{\text{old}}(y_{ik} \mid x, y_{i, < k})}. \quad (4)$$

Define the clipped token surrogate and reference-regularized objective by:

$$\ell_{ik}(\theta) = \min(\rho_{ik} A_i, \text{clip}(\rho_{ik}, 1 - c, 1 + c) A_i), \quad (5)$$

$$J(\theta) = \mathbb{E} \left[\frac{1}{g} \sum_{i=1}^g \frac{1}{L_i} \sum_{k=1}^{L_i} \{ \ell_{ik}(\theta) - \beta D_{\text{KL}}(\pi_{\theta}(\cdot | h_{ik}) \| \pi_{\text{ref}}(\cdot | h_{ik})) \} \right]. \quad (6)$$

Here $h_{ik} = (x, y_{i,<c})$, $0 < c < 1$ is the clipping parameter, and $\beta \geq 0$ controls regularization against the fixed SFT reference policy. The expectation is over training contexts and candidate groups sampled from the rollout policy. The exact KL above defines the objective; a practical sampled estimator must be specified and logged rather than silently equated with exact computation. Only generated tokens receive policy-gradient loss; prompts and tool outputs are masked out. Constant-reward groups may produce a KL update. They need not do so: when the current and reference policies coincide, the exact KL gradient is zero. Other optimizer state can also affect an update. Mixed-group frequency measures availability of reward signal, not successful learning.

The response-length normalization is a deliberate baseline choice, not a claim of an unbiased optimizer. Existing work questions GRPO’s length and difficulty effects [16]. Record length distributions and use an explicitly identified normalization variant only if chosen on development data before final evaluation. A backend that implements a different aggregation or KL estimator must report that difference.

The initial rollout group size is a pilot parameter. LoRA rank, target modules, learning rate, context limit, temperature, token cap, number of updates, and checkpoint-selection rule are fixed after profiling and before final evaluation. They are not reported as completed settings in this manuscript. NeMo RL documentation establishes available tooling, not that a particular Nemotron configuration will fit the project’s hardware [13, 14].

4.4 Acceptance is independent of optimization

At inference, the runner accepts a candidate only if its structural checks and every mandatory development case pass. It does not accept the highest-reward patch when all patches fail. Model-generated confidence is not an acceptance criterion. Hard gates also apply to non-RL comparators unless an explicitly isolated ablation changes them.

Only one repair job per application version may promote at a time. Promotion uses a compare-and-swap on the expected parent identifier and stores the tested code, dependency, policy, fixture, and evaluator hashes. A stale candidate must be revalidated against the new parent rather than overwriting a later repair. Pin each demo session to an immutable code version. Contract or dependency changes invalidate the old validation scope and require revalidation; the finite-suite observation cannot silently span such changes.

Historical memory contains cases that an accepted version has actually passed. A failed new incident remains in an unresolved-incident registry and does not become a falsely certified historical success. Evaluation reports unresolved incidents even if later work proceeds. The system makes no statement that all known problems have been fixed merely because a new version is accepted.

4.5 Proposed repair procedure

```

Inputs: parent p, frozen repair policy, validated incident e,
        protected development checks D, retained cases H, budget B
Replay e on p using trusted observations.
If replay is incomplete: record indeterminate outcome; return p.
If the contract already holds:
    Validate D and H on p; if they pass, retain e as already contained.
    Otherwise record the failing checks; make no new success claim.
    Return p without spending a generation call.
For attempt = 1 ... B:
    Generate a patch from bounded context and allowed diagnostics.
    Reject forbidden source edits; run the candidate in isolation.
    Execute D, H, and e; require complete, trusted outcomes.
    If all mandatory checks pass:
        Atomically activate the exact tested artifact; retain e.
        Return the accepted candidate and its evidence.
    Record failure and provide only permitted diagnostics.
Record e as unresolved; return p without a repair-success claim.
    
```

An incident is independently validated on its declared baseline before entering the stream. Failure to reproduce on a later version can mean a previous patch already contained it; that is distinct from an infrastructure failure or an unreproducible stochastic conversation. Report already-contained, newly repaired, unresolved, and indeterminate cases separately. Already-contained cases count toward final containment but not newly generated repair success.

This is specification pseudocode, not an existing executable implementation. The runner must enforce the same declared resource ceilings across repair methods and record actual consumption. Checkpoint selection and final benchmarking are separate operations.

5 Limits of validation

Under deterministic replay, fixed test semantics and dependencies, trusted observations, and atomic activation of the tested artifact, requiring every retained case to pass preserves that finite suite by induction. This elementary property follows from the gate. A program can still pass those cases and fail on an untested input. Thus admission establishes observed test satisfaction within a pinned scope, not general security. Hidden companion cases evaluate generalization beyond the cases enforced by the gate.

6 Research questions and experimental design

6.1 Staged questions and decisions

The first study is an exploratory usefulness pilot. It compares frozen feedback repair with frozen independent sampling under common limits and with direct enforcement under the same policy and state access. Its deliverable is a rerunnable set of transitions, patches, outcomes, and costs from two independently authored integrations. It establishes local behavior and integration effort; two implementations do not support a population-level transfer claim.

The later learning study asks whether SFT followed by RL improves held-out repair over the exact SFT checkpoint. Continued SFT or iterated rejection-sampling fine-tuning at declared comparable collection and training budgets tests the stronger claim that RL is a better use of those resources. A larger hosted repair model is a practical comparator, not the causal control for training a smaller checkpoint.

Question	Required comparison	Evidence and decision
Does repair add practical value?	Feedback repair, independent sampling, direct enforcement, reference guard	Per-implementation security, utility, effort, and cost; narrow the product if enforcement dominates
Does additional RL improve repair?	Matched SFT start with and without RL	Held-out lineages, paired outcomes, training-seed uncertainty
Is RL preferable to extra imitation?	Continued SFT or iterated RFT at comparable budgets	Separate effects of optimizer choice, extra samples, and extra compute

For the conditional learning study, define differences as SFT-plus-RL minus SFT. A favorable confirmatory result requires an upper one-sided 97.5% confidence bound below zero for composite security failure, and a lower one-sided 97.5% bound above minus delta for clean success. Both must pass. Delta is a harm-justified utility margin fixed before final access. Appendix D illustrates precision costs; it does not select the final sample size. If precision or independent family counts are inadequate, label the comparison exploratory. Fewer incomplete runs alone cannot establish fewer observed violations.

Freeze the sampling unit, interval procedure, joint-power target, sample size, seeds, and stopping rule after calibration and before final scoring. An inconclusive comparison remains inconclusive.

6.2 Applications, acquisition, and the first pilot

Use a refund integration and a separately authored scoped-file integration. The initial target is three applicable fault classes: ownership/scope validation, replay/idempotency, and a contract-defined state precondition. Each implementation gets its own semantics; do not force a class that is inapplicable. One fixed short incident stream per implementation supports limited snapshot comparisons. Order sensitivity, adaptive attackers, public benchmark adaptations, and external developer studies are later extensions.

Acquire the first integrations from separate implementations with documented provenance. A second author must record shared templates and libraries rather than claiming independence from separate filenames. The inventory records author/reviewer, source and license, contract, shared ancestors, seed defect and descendants, fixture IDs, reference repair, and intended partition. Reference repairs remain outside model inputs. Any later corpus expansion needs a family-count target derived from pilot variance and the intended transfer claim.

The pilot’s four main conditions are unrepaired, frozen independent-sampling repair, frozen feedback repair, and direct enforcement. A handwritten reference guard checks fixture solvability; record its human effort separately. Both repair conditions receive the same original source, incident, full contract, and allowed historical context. Independent sampling gets no intermediate execution diagnostics; feedback repair may condition later patches on declared development diagnostics. Both use the same acceptance checks, sealed outcomes, model, and prespecified resource ceilings. Feedback is the treatment, so initial information is matched while intermediate observations intentionally differ.

Declare candidate, total input/output token, test-execution, and wall-time ceilings before scoring. The feedback arm permits at most three attempts; the independent arm samples up to the same candidate ceiling from the initial context. Use a fixed candidate order and choose the first candidate passing development gates. Unused allowance is recorded, not padded with dummy calls. An exhausted budget, rejected patch, or failed repair stays in the denominator.

Each adapter must have an explicit contract, clean fixtures, and independently reviewed legitimate workflows. Fault injection supplies controlled incidents such as missing ownership checks, duplicate

operations, or incomplete state validation. Faults, seed programs, and reference repairs are versioned. The target agent must actually be capable of issuing the relevant action, and the injected fault must reproduce. Naturally occurring defects, if later included with authorization, are reported separately from injected ones.

The primary applicability claim concerns integrations with editable tool adapters and reliable oracles. An AgentDojo adaptation is supplementary only where it preserves the original task and attacker semantics [1]. If the adaptation changes tools, state, or scoring, it is named as a derived benchmark. InjecAgent can supply additional relevant test scenarios but does not automatically provide a drop-in code-repair environment [2].

6.3 Data partitions and leakage prevention

The study uses four logical partitions: training incidents and reward cases; development validation for method and checkpoint selection; final incident streams used as evaluation inputs; and sealed outcome cases used only for scoring. A final incident is necessarily shown to a repair method, but its hidden variants and legitimate outcome cases are not. This distinction allows evaluation-time repair without training on final outcomes.

Split by implementation lineage, contract instantiation, and attack family. Keep paraphrases, mutations of one seed bug, and alternate identities derived from one scenario together. Hold out entire implementations to test transfer. Within-domain transfer and leave-one-domain-out transfer are different experiments and receive different labels.

No final score selects a checkpoint, reward coefficient, prompt, or patch. During sequential repair, save program snapshots and evaluate them afterward; do not feed final outcomes into subsequent repairs. Public benchmark training contamination may remain unknowable, so fresh independently authored final cases are needed and their limitations disclosed.

6.4 Comparison conditions

Condition	Purpose
Unrepaired application	Establish that the incident occurs and characterize original utility
Frozen-model one-shot or best-of-budget repair	Test whether repeated sampling already solves the task
Frozen-model feedback repair	Product baseline with the same gate and historical suite
Supervised fine-tuning plus the same repair loop	Primary learning comparator: the exact pre-RL checkpoint
The matched SFT checkpoint followed by RL	Proposed primary learning treatment
Direct enforcement at the trusted broker	Test whether the available checker already solves the problem
Manually implemented reference guard	Check fixture solvability and the value of repair automation

The frozen, supervised, and RL repairers share the same base checkpoint, tool access, contracts, context limits, final gate, and inference limits where technically possible. The target agent model remains identical across conditions. A larger Token Factory model can be an additional practical baseline but cannot isolate the causal effect of RL training on a smaller local model.

The primary study uses an SFT checkpoint as the common initialization for the SFT-only and SFT-plus-RL conditions. Add a continued-SFT or rejection-sampling control with a declared data-generation and compute budget to test whether extra successful examples explain any improvement.

Report the original frozen model separately. A direct-from-base RL run is an exploratory variant, not a replacement for this contrast.

The supervised condition trains on successful patches from a declared collection procedure. Compare against rejection-sampling fine-tuning using candidate pools of comparable provenance, and disclose any differences in samples, test executions, or training compute. Match inference budgets and separately report total training and data-generation costs. A human reference is neither an automated baseline with equal labor nor a proof that the reference is flawless.

Relevant runtime defenses, including AgentSpec and compatible training-free policy methods, should be evaluated when making broad defense claims [7, 9]. If implementations or semantic compatibility are unavailable, narrow the claims rather than asserting superiority over the field.

6.5 Ablations and sequential evaluation

Separate memory in the model context, memory in the training reward, and memory in the acceptance gate. Removing a summary from the prompt while retaining the full gate is not the same experiment as removing historical enforcement. The primary reward-memory ablation removes historical checks from the binary training reward while retaining the same evaluation-time gate. Diagnostic pass fractions remain logged. A gate-removal ablation is conducted only in the isolated harness and reported separately.

For each held-out application, reveal a fixed incident stream. Each method starts from the same parent code and maintains its own version history. Use the same incident order across paired methods; repeat with prespecified orders to measure order sensitivity. A method can fail to repair an incident, and that failure remains in the denominator.

Historical retention is measured on a fixed shared panel of sealed companion cases from earlier failure families. Report the unconditional pass rate on that same panel at each code version. Also report the fraction of previously passed companions that later fail, with method-specific numerators and denominators. The conditional subsets differ across methods, so their forgetting rates alone are not a fair cross-method comparison. All snapshot scoring happens after the stream is complete; no final outcome informs subsequent repair.

6.6 Deferred adaptive-attack study

The first pilot uses identical static held-out cases across methods. In a later separately locked study, adaptive evaluation uses a fresh, independent testing process against each frozen defender version, with the same permitted observations and per-target query budget. An adaptive attacker’s final probes may depend on its target’s responses; the attack-selection rule and query budget are fixed, while the repairer remains frozen and receives no patch feedback from this evaluation.

Compare outcomes under these two settings separately. Do not call a fixed list of paraphrases an adaptive evaluation. Existing work such as PISmith informs the choice of adaptive comparators, subject to authorized sandbox use and implementation availability [10]. Training an increasingly capable attacker is optional future work, not required to establish whether repair-policy learning helps.

7 Measurement and statistical analysis

For each attack episode, record whether any unauthorized state transition occurred, whether the agent proposed an unauthorized action, whether the legitimate task completed, and whether execution finished. Once a violation occurs, a later crash does not erase it. A timed-out episode with incomplete observations is not certified safe.

Define the primary composite endpoint F as one when an episode contains an observed violation or has incomplete security observations, and zero otherwise. Call it security failure or incomplete execution, not executed violation. Let n be the scheduled episode count, v the number with an observed violation, and u the number without an observed violation whose security outcome is unknown. Report the observed lower bound v/n and conservative upper bound $(v+u)/n$, alongside completion coverage. These are missing-outcome bounds on this scheduled sample, not population confidence intervals. A known policy violation remains in v even if that episode later times out.

Measure	Operational definition
Observed executed violations	v/n with explicit missing-outcome bounds; do not relabel unknown outcomes as observed violations
Security failure or incomplete execution	$(v+u)/n$, the prespecified primary composite endpoint
Unsafe proposal rate	Episodes containing an unauthorized proposed action, regardless of whether execution was rejected
Clean task success	Legitimate episodes that complete the required state outcome without a violation, divided by all scheduled clean episodes
Secure task success under attack	Attack episodes that finish the legitimate task without a violation, divided by all scheduled attack episodes
Incident outcomes	Already contained, newly repaired, unresolved, and indeterminate counts; final containment uses all scheduled validated incidents
Historical-family retention	Pass rate on a fixed common panel; conditional regression is supplementary with its own denominator
Repair efficiency	Calls, tokens, test executions, wall time, and billed cost, including unsuccessful attempts
Operational feasibility	Valid-patch rate, acceptance rate, completion coverage, and zero-advantage training-group frequency

Repair effectiveness and target-agent behavior are measured on separate tracks. Deterministic action replay isolates adapter behavior; repeated end-to-end runs expose stochastic choices and changed observations. Success on fixed action replay is not a substitute for end-to-end utility.

Use paired task instances, fixtures, and incident streams. Report results per application and domain before aggregate summaries. Estimate uncertainty by resampling at the highest genuinely independent unit relevant to the claim, normally held-out implementation or independently authored scenario family; preserve nested repeated runs within that unit. Training-seed variability is an additional level and is shown separately or in a prespecified hierarchical analysis. Three application domains with many paraphrases do not create hundreds of independent replications.

The two-implementation pilot reports each integration separately and cannot support a bootstrap claim about a population of implementations. A larger calibration study estimates variance, effect sizes of interest, and cost for sample-size planning. Those estimates are not final results. Freeze the primary comparator, harm-justified non-inferiority margin, confidence procedure, sample size, maximum runs, and exclusion policy before opening final outcomes. Plan power for the joint success criterion rather than treating two individual power targets as joint power. Use at least several independent training seeds when feasible; if only one seed is affordable, label the RL comparison exploratory. Do not repeatedly inspect significance and extend the study until it succeeds.

Evaluator failures, external outages, and candidate-induced crashes require distinct labels. Predeclare a bounded infrastructure retry policy and retain original attempts in the audit trail. Missingness must be reported per condition; favorable safety claims cannot rest on one method failing to execute more often. Secondary comparisons use multiplicity adjustment or are clearly labeled exploratory.

8 Implementation and compute plan

The proposed product comprises a Python integration package, one API service, a repair worker, isolated executors, a small relational store, and versioned artifacts. The application owner supplies reset, snapshot, task, and transition evaluation hooks plus editable paths. Integration therefore requires more than a chatbot URL.

The product baseline can use NVIDIA Nemotron inference through Nebius Token Factory, while the application and disposable execution workers run on Nebius infrastructure. Nebius documents Nemotron 3 Super availability for coding and agentic use [15]. Record the actual model identifier and provider revision where exposed rather than assuming a fixed identifier from a product name.

Weight training is a separate Nebius AI Cloud workload with model weights, optimizer state, and rollout infrastructure. Token Factory inference requests do not by themselves update model weights. A training-compatible Nemotron checkpoint and NeMo RL configuration must pass an actual end-to-end pilot before the RL study is scheduled. Parameter-efficient training reduces trainable state but does not eliminate base-model or rollout memory requirements [13, 14].

A concrete candidate to profile is NVIDIA-Nemotron-3-Nano-30B-A3B-Base-BF16, with a common SFT warm start for both learning conditions. NVIDIA publishes a Nano GRPO-LoRA recipe with two nodes and eight GPUs per node [27]. This is an example configuration, not a minimum hardware requirement or our allocated budget. Pin a framework commit and full inherited configuration before execution; the published recipe includes architecture-specific LoRA exclusions.

The training gate requires rollout, protected scoring, optimizer update, checkpoint save, reload, and fresh generation from the saved artifact. Log finite losses, trainable-weight changes, per-task reward variation, several steady-state step profiles, and total resource use. Set a monetary ceiling before allocation. Export and serve the exact artifact before claiming deployment feasibility. Hosted custom-adaptor support must be demonstrated; a documentation error or generic custom-weights feature proves neither compatibility nor incompatibility.

The target agent and repair policy are separate roles. A Nano repair-policy result supports claims about that tested repairer and application setup; it does not establish a training effect on Super. A trained repairer may be served separately while the target model stays fixed.

Implementation proceeds through reproducible baseline failure, successful generated repair, protected evaluation, independent integration, trace collection, and only then training. A practical experiment must establish a nontrivial rate of valid candidates and varying rewards before committing to a large run. No project hardware allocation, training duration, token price, or expected accuracy is asserted.

Report data-generation cost, training GPU-hours, evaluation cost, and per-repair inference cost separately. If C-train includes all incremental collection, training, and validation cost and the measured per-repair saving is positive s , a simple cost-only amortization estimate is C-train divided by s repairs. This estimate is conditional on comparable repair quality and stable workload; it is not a business forecast.

9 Reproducibility and evidence release

The release should include application source and licenses, independent contract checkers, fixture and split manifests, bug provenance, model and tokenizer identifiers, dependency locks, training configurations, seeds, reward implementation, accepted and rejected patches, complete scheduled-episode outcomes, and analysis code. Reference patches are withheld from model inputs. Secrets and private customer data are excluded.

Each candidate record binds its parent source hash, patch hash, fixture, checker version, test suite, generation configuration, token accounting, and promotion decision. Each training record additionally binds the checkpoint, rollout policy, reward components, and group identifier. A separate final-evaluation manifest identifies which artifacts were frozen before outcomes were opened.

Recorded model-output replay reproduces historical tool execution. Fresh inference runs estimate repeatability and can differ even under nominally identical settings. Both forms of reproduction are useful and must be labeled. Cryptographic hashes establish artifact identity, not the correctness of the observations or claims.

There is currently no released implementation, dataset, checkpoint, experiment log, repository URL, or executable reproduction command for this proposal. Such artifacts must be added after they exist. An empirical submission requires an independent rerun on the released commit and a check that every result is generated from the accompanying raw data. Ship verification scripts with relative paths, pinned dependencies, and the outputs they verify; a link to a private notes directory is insufficient.

10 Limitations, failure criteria, and ethics

The strongest objection is that the trusted evaluator may already contain the solution. Converting its predicates to direct enforcement can dominate the repair approach in the initial simulator. Novelty remains unestablished, and there is no evidence yet that the integration or data-collection burden is justified. Application authorization should be implemented correctly without requiring a learning system. For a small stable application, a manually written guard may be simpler and more reliable. PATCHLOOP must demonstrate reduced repair effort, useful transfer, or better performance under changing adapters to justify its additional machinery. A negative result is acceptable and must narrow the product claim.

Synthetic fault injection risks making the problem artificially easy or tailored to the reward. Independent implementations, natural defects when available, and separately authored tests reduce this risk without eliminating it. Owner-authored oracles may themselves be incomplete or wrong. Reward maximization can exploit those gaps, and the method cannot learn a missing requirement simply because the evaluator is described as protected.

The method also risks selection overfitting from repeated development feedback, dependence on particular model families, insufficient independent applications, and inadequate compute to distinguish RL from ordinary successful-example training. The proposed hard gate may reject every useful partial repair. Conversely, a permissive gate can admit brittle code. These are study outcomes to report, not problems to hide by changing evaluation rules after inspecting results.

The study uses developer-authorized environments and synthetic state. The repair process cannot access live payment credentials, personal files, or external production accounts. Public visitors may challenge provided fixtures but cannot upload arbitrary executable repositories into the hosted demo. Production changes remain reviewed patches. Any naturally occurring vulnerability discovered in an external project follows coordinated disclosure rather than automatic publication of a live exploit.

AI-assisted drafting and coding should be disclosed according to the eventual venue’s policy. Human authors remain responsible for source verification, mathematical statements, experimental integrity,

and the final submission. No peer-review acceptance, human evaluation, or external developer study is asserted.

11 Current evidence and conclusion

This protocol specifies a staged test of usefulness and learning. The first release should establish trusted execution and controlled frozen-model comparisons. Transfer, unseen-task utility, adaptive robustness, and cost effectiveness remain open empirical questions.

Proceed to learning only after evaluator validation, independent data collection, reward-signal measurement, and a compute smoke test. Report negative and inconclusive findings with their scope. Submission format, anonymity, and acceptance depend on the eventual venue; this protocol predicts no review outcome.

References

- [1] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*. arXiv:2406.13352, version 3. <https://arxiv.org/abs/2406.13352>
- [2] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. *InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents*. Findings of ACL 2024, pp. 10471-10506. DOI: 10.18653/v1/2024.findings-acl.624. <https://aclanthology.org/2024.findings-acl.624/>
- [3] Zheng Yu, Ziyi Guo, Yuhang Wu, Jiahao Yu, Meng Xu, Dongliang Mu, Yan Chen, and Xinyu Xing. 2025. *PATCHAGENT: A Practical Program Repair Agent Mimicking Human Expertise*. USENIX Security 2025, pp. 4381-4400. <https://www.usenix.org/conference/usenixsecurity25/presentation/yu-zheng>
- [4] Nafis Tanveer Islam, Mohammad Bahrami Karkevandi, and Peyman Najafirad. 2024. *Code Security Vulnerability Repair Using Reinforcement Learning with Large Language Models*. arXiv:2401.07031, version 2. Preprint citation. <https://arxiv.org/abs/2401.07031>
- [5] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. 2025. *SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution*. arXiv:2502.18449, version 2. <https://arxiv.org/abs/2502.18449>
- [6] Yuxiang Wei, Zhiqing Sun, Emily Mcmilin, Jonas Gehring, David W. Zhang, Gabriel Synnaeve, Daniel Fried, Lingming Zhang, and Sida Wang. 2026. *Toward Training Superintelligent Software Agents through Self-Play SWE-RL*. Proceedings of ICML 2026, PMLR 306, pp. 134075-134095. <https://proceedings.mlr.press/v306/wei26x.html>
- [7] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2026. *AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents*. ICSE 2026, pp. 2938-2950; author version arXiv:2503.18666v3. <https://arxiv.org/abs/2503.18666>
- [8] Lipeng He, Yihan Wang, Jiawen Zhang, and N. Asokan. 2026. *Defending against Adaptive Prompt Injection Attacks via Reasoning-enabled Task Alignment*. arXiv:2606.15441. Preprint. <https://arxiv.org/abs/2606.15441>

- [9] Minh Nhat Le, Nisarga Gondi, Yibo Peng, Ronghao Ni, Limin Jia, Beidi Chen, and Haizhong Zheng. 2026. *Self-Evolving Defense: Continual Security Policy Learning for LLM Agents*. arXiv:2609.36603. Preprint submitted 29 September 2026. <https://arxiv.org/abs/2609.36603>
- [10] Chenlong Yin, Runpeng Geng, Yanting Wang, and Jinyuan Jia. 2026. *PISmith: Reinforcement Learning-based Red Teaming for Prompt Injection Defenses*. arXiv:2603.13026v2; author record states to appear in COLM 2026. <https://arxiv.org/abs/2603.13026>
- [11] Mickel Liu, Liwei Jiang, Yancheng Liang, Simon Shaolei Du, Yejin Choi, Tim Althoff, and Natasha Jaques. 2026. *Chasing Moving Targets with Online Self-Play Reinforcement Learning for Safer Language Models*. Proceedings of ICML 2026, PMLR 306, pp. 76897-76930. <https://proceedings.mlr.press/v306/liu26bn.html>
- [12] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. arXiv:2402.03300v3. <https://arxiv.org/abs/2402.03300>
- [13] NVIDIA. *An In-depth Walkthrough of GRPO in NeMo RL*. Official documentation, accessed 3 October 2026. <https://docs.nvidia.com/nemo/rl/nightly/guides/grpo.html>
- [14] NVIDIA. *LoRA (Low-Rank Adaptation), NeMo RL*. Official documentation, accessed 3 October 2026. <https://docs.nvidia.com/nemo/rl/nightly/guides/lora.html>
- [15] Nebius. *NVIDIA Nemotron 3 Super now available on Nebius Token Factory*. Official product announcement, accessed 3 October 2026. <https://nebius.com/blog/posts/nemotron3-super-now-available>
- [16] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. 2025. *Understanding R1-Zero-Like Training: A Critical Perspective*. arXiv:2503.20783v2. <https://arxiv.org/abs/2503.20783>
- [17] Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Peter Belcak, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, Yejin Choi, Jan Kautz, and Pavlo Molchanov. 2026. *GDPO: Group reward-Decoupled Normalization Policy Optimization for Multi-reward RL Optimization*. arXiv:2601.05242. NVIDIA technical report/preprint. <https://arxiv.org/abs/2601.05242>
- [18] Wei Zhao, Zhe Li, Peixin Zhang, and Jun Sun. 2026. *ClawGuard: A Runtime Security Framework for Tool-Augmented LLM Agents Against Indirect Prompt Injection*. arXiv:2604.11790v2. <https://arxiv.org/abs/2604.11790>
- [19] Yu He, Haozhe Zhu, Yiming Li, Shuo Shao, Hongwei Yao, Zhihao Liu, and Zhan Qin. 2026. *AttriGuard: Defeating Indirect Prompt Injection in LLM Agents via Causal Attribution of Tool Invocations*. USENIX Security 2026, pp. 1547-1566. <https://www.usenix.org/conference/usenixsecurity26/presentation/he-yu>
- [20] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. *Tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains*. arXiv:2406.12045. <https://arxiv.org/abs/2406.12045>
- [21] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. *Teaching Large Language Models to Self-Debug*. arXiv:2304.05128v2. <https://arxiv.org/abs/2304.05128>

- [22] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. *Reflexion: Language Agents with Verbal Reinforcement Learning*. arXiv:2303.11366v4. <https://arxiv.org/abs/2303.11366>
- [23] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024; author version arXiv:2310.06770v3. <https://arxiv.org/abs/2310.06770>
- [24] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. *Agentless: Demystifying LLM-based Software Engineering Agents*. arXiv:2407.01489v2. <https://arxiv.org/abs/2407.01489>
- [25] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. 2022. *CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning*. arXiv:2207.01780v3. <https://arxiv.org/abs/2207.01780>
- [26] Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Quentin Carbonneaux, Taco Cohen, and Gabriel Synnaeve. 2025. *RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning*. arXiv:2410.02089v2; first submitted 2024. <https://arxiv.org/abs/2410.02089>
- [27] NVIDIA. *Nano v3 GRPO-LoRA FSDP2 example configuration*. NeMo RL repository, inspected 4 October 2026. Mutable example; a run must pin its commit. <https://github.com/NVIDIA-A-NeMo/RL/blob/main/examples/configs/recipes/llm/grpo-nanov3-30BA3B-2n8g-fsdp2-lora.yaml>

A Evidence required before an experimental submission

Claim	Required evidence	Current status
The repair loop works	Released code, failing fixture, real generated patch, independent replay	Not implemented or measured
RL changes the repair policy	Training logs, checkpoint hashes, optimizer and rollout configuration	Not trained
RL improves transfer	Paired held-out implementation results against frozen and supervised baselines	Not evaluated
Legitimate utility is preserved	Prespecified non-inferiority analysis with adequate precision	Not evaluated
Previous fixes generalize	Sequential results on sealed historical-family companions	Not evaluated
The method is cost effective	Complete collection, training, failed-repair, and inference accounting	Not measured
Finite retained cases remain passed	Section 5 assumptions plus implementation evidence	Conditional invariant only
The system is universally secure	Not supported by finite tests or the proposed method	No such claim

B Protocol lock record

Before final evaluation, timestamp and hash one manifest containing: source and checker commits; implementation-family split; exact incident streams; target and repair model identities; all training settings; supervised-data provenance; inference and attacker budgets; candidate-selection rules; mandatory suites; primary contrast and utility margin; independent-unit definition; sample-size justification; confidence procedure; seed list; outage/retry rules; stopping rule; cost measurement; and permitted exclusions. The current manuscript is a proposed protocol, not evidence that this lock has already occurred.

C Reference audit

References [1]-[12] were checked against author-hosted arXiv records or official proceedings. Full method text was additionally inspected for GRPO [12], RETA [8], and PISmith [10]; the narrower overlap statements for other works are supported by their abstracts and metadata. References [13]-[15] are official implementation or product sources. References [16]-[17] were checked against author arXiv records for their optimization concerns. References [18]-[26] were checked against author abstracts and official proceedings for the narrow overlap statements; reference [27] is an inspected official configuration. These checks establish existence and stated scope, not independent reproduction. No numerical result from another paper is reused as a PATCHLOOP result. This audit verifies the existence and stated scope of sources; it is not an independent reproduction or a complete systematic literature review.

D Illustrative precision calculations

These are normal-approximation planning calculations, not measurements or a final sample-size justification. Assume independent arms, 80% marginal power, no continuity correction, and quantiles $z_{0.975}$ and $z_{0.8}$. For utility non-inferiority with both true success rates equal to $p = 0.9$ and one-sided alpha 0.025:

$$n \simeq \frac{2p(1-p)(z_{0.975} + z_{0.8})^2}{\delta^2}. \quad (7)$$

Target	Episodes per arm	Interpretation
Security failure 0.40 to 0.25	152	Two-sided alpha 0.05 approximation; marginal superiority power
Utility margin 0.02	3532	Assumes equal true success at 0.90
Utility margin 0.05	566	Same assumptions
Utility margin 0.10	142	Would tolerate a rise in failure from 0.10 to 0.20 at the boundary

Under the equal-rate paired normal approximation, variance is multiplied by $1 - \rho$, where ρ is within-pair outcome correlation; pairing alone guarantees no reduction. Cluster dependence, unequal cluster sizes, lineage structure, training seeds, and missing outcomes require a design-specific analysis. Two tests with marginal power 0.8 have joint power between 0.6 and 0.8; independence gives 0.64. Choose the utility margin from tolerable harm, then budget for the required precision. If it is unaffordable, report utility descriptively. The source bundle includes a dependency-free script reproducing this arithmetic.

E Revision record and unresolved decisions

Version 0.3 integrates the verified critical review into the method and product plan. It introduces a two-implementation usefulness pilot, separates initial information from intermediate feedback, adds evaluator acceptance tests and authorship provenance, expands related work, gives a labeled ownership example and architecture figure, states the exact cost attenuation and conditional KL behavior, and adds reproducible planning arithmetic. Elementary testing observations are compressed into Section 5.

The remaining decisions require implementation: evaluator integrity, usefulness against direct enforcement, independently sourced training data, measured reward variation, hardware and serving compatibility, and a calibrated final design. The source package reproduces document compilation and illustrative calculations; it does not reproduce an unperformed PATCHLOOP experiment.